

FUNDAMENTALS OF ARTIFICIAL INTELLIGENCE

SESSION 1_01.09.2024

Module-1 Introduction to Python for Data Science

- Python Programming Introduction
- Python Data structures and Functions
- Basic Python programs
- Introduction to Data science
- Understanding the nature of Data -Types of Data
- Introduction to libraries:
 - NumPy, Pandas, Matplotlib
- Data types in Python relevant to data science
- Applications using Python libraries
- Hands-on practice with NumPy and Pandas.

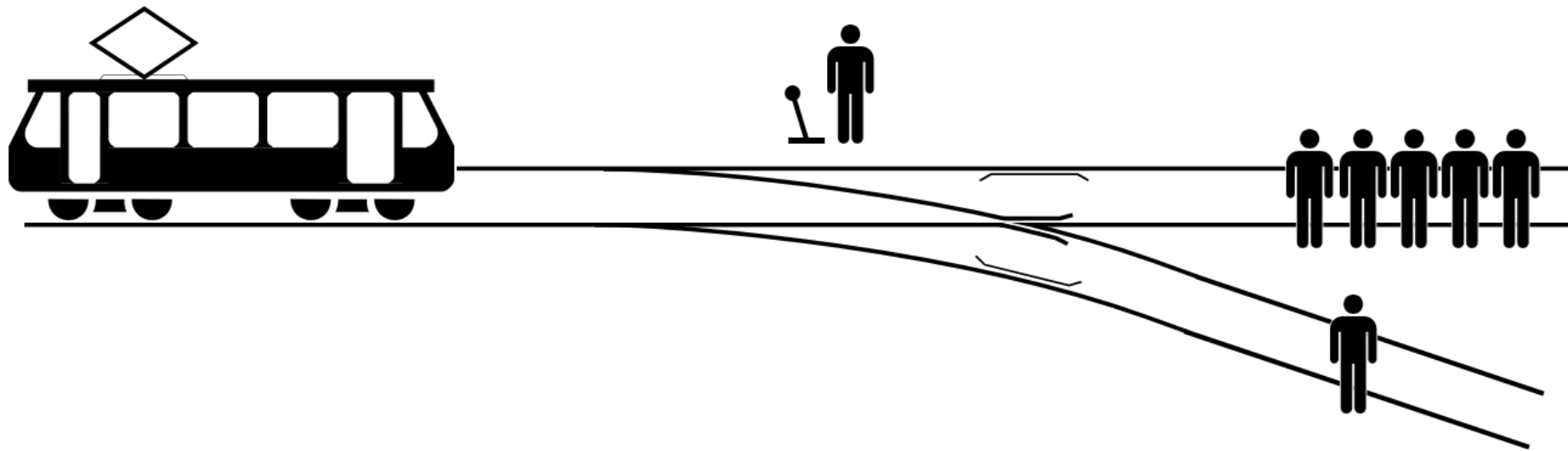
Practical Session

Lab 1 Python Basics and Control Structures

Lab 2 Create a NumPy array and perform basic operations

Lab 3 Use Pandas to load a CSV file, display the first 10 rows, and perform basic data manipulation (sorting, filtering).

Artificial Intelligence



Artificial Intelligence



Artificial Intelligence



Blockchain
APPS DEVELOPER

Role Of AI in **Self-Driving** **Cars**

+91 9489606634 BlockN_Bitz Block'N Bitz support@blockchainappsdeveloper.com

Introduction to Data science

Data Science includes data gathering, analysis and decision-making.

Data science is a multidisciplinary field that uses statistical and computational methods to extract insights and knowledge from data.

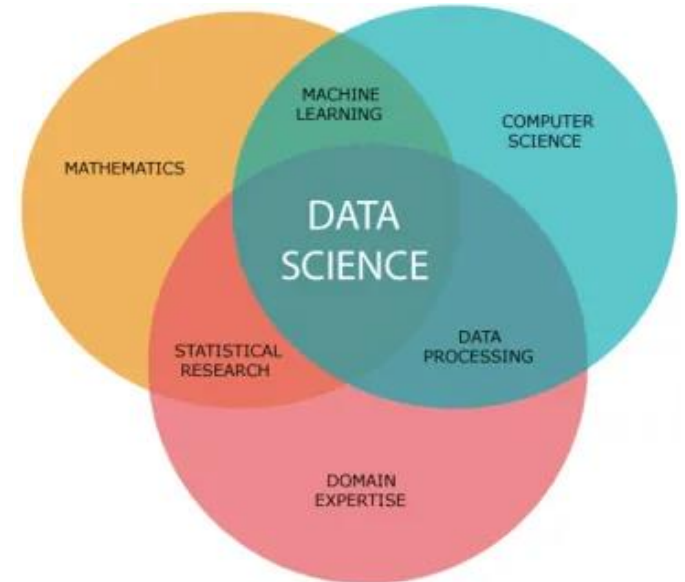
By using Data Science, companies are able to make:

- Better decisions (should we choose A or B)
- Predictive analysis (what will happen next?)
- Pattern discoveries

(find pattern, or maybe hidden information in the data)

A Data Scientist requires expertise in several backgrounds:

- Machine Learning
- Statistics
- Programming (Python or R)
- Mathematics
- Databases



Applications of Data Science

- For route planning: To discover the best routes to ship
 - To foresee delays for flight/ship/train etc. (through predictive analysis)
 - To create promotional offers
 - To find the best suited time to deliver goods
 - To forecast the next years revenue for a company
 - To analyze health benefit of training
 - To predict who will win elections
- Consumer goods
 - Stock markets
 - Industry
 - Politics
 - Logistic companies
 - E-commerce
 - Search Engines
 - Transport
 - Finance
 - E-Commerce
 - In Health Care

Work of a Data Scientist:

1. Ask the right questions - To understand the business problem.
2. Explore and collect data - From database, web logs, customer feedback, etc.
3. Extract the data - Transform the data to a standardized format.
4. Clean the data - Remove erroneous values from the data.
5. Find and replace missing values - Check for missing values and replace them with a suitable value (e.g. an average value).
6. Normalize data - Scale the values in a practical range (e.g. 140 cm is smaller than 1,8 m. However, the number 140 is larger than 1,8. - so scaling is important).
7. Analyze data, find patterns and make future predictions.
8. Represent the result - Present the result with useful insights in a way the "company" can understand.



Common challenges in Data Science

Data quality issues (like incomplete or noisy data),

Managing and processing large datasets efficiently (big data challenges),

Selecting the right algorithms and techniques for specific problems, and

Ensuring ethical considerations around data privacy and bias are addressed.

Communicating findings in a way that stakeholders can understand and act upon is crucial for the success of Data Science initiatives.

Understanding the nature of Data -Types of Data

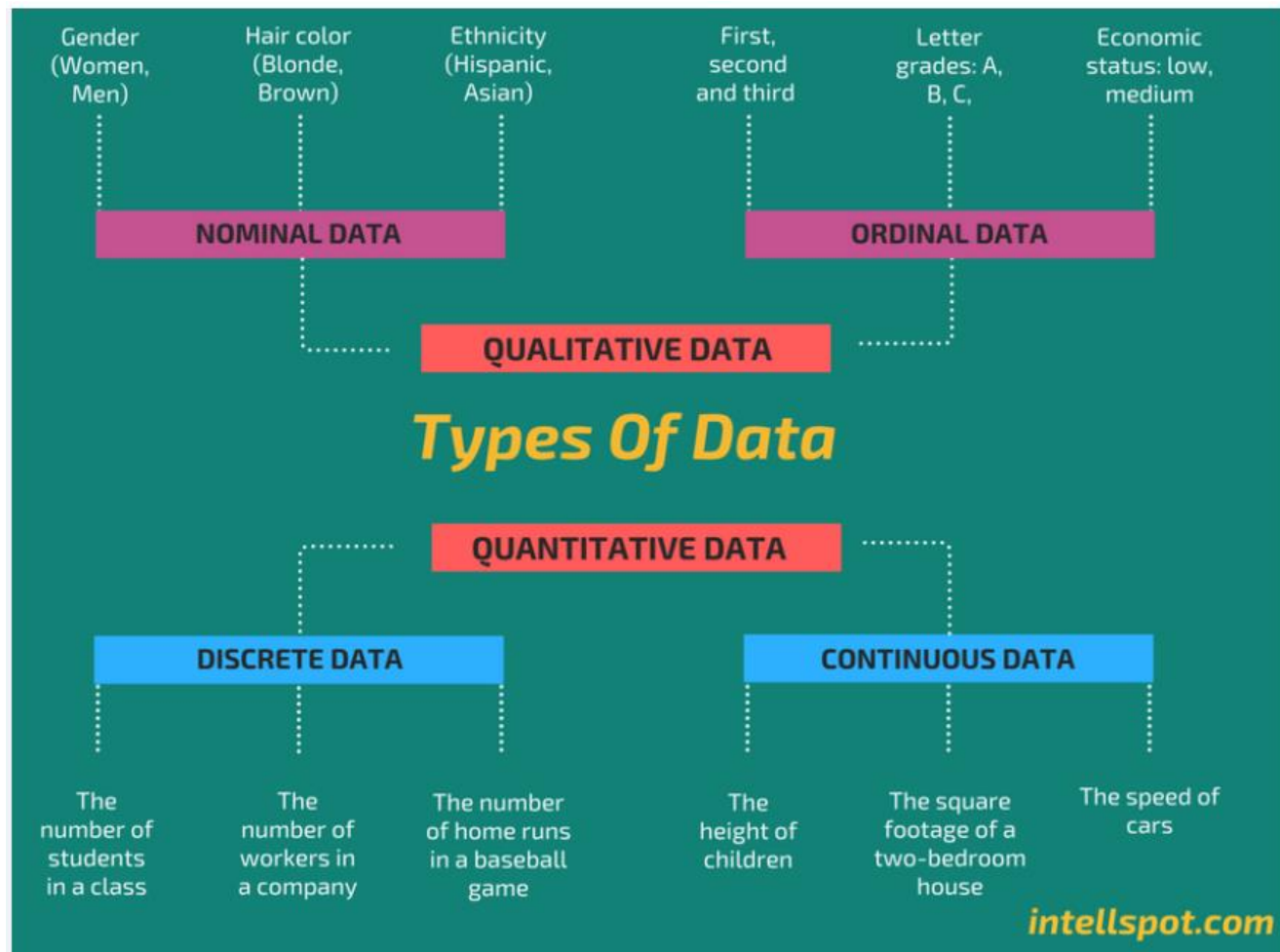
By nature, data are either quantitative or qualitative.

Quantitative data are numerical and qualitative data are descriptive.

It is possible to transform qualitative data into numerical values.

In sciences, data can also be graphic in nature.

Understanding the nature of Data -Types of Data



Why Python?

- Works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Has a simple syntax similar to the English language.
- Has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Can be treated in a procedural way, an object-oriented way or a functional way.
- Python can be written in a text editor.
- It is possible to write Python in an Integrated Development Environment, such as Thonny, Pycharm, Netbeans or Eclipse which are particularly useful when managing larger collections of Python files.

Python Programming Introduction

Created by Guido van Rossum, and released in 1991.

It is used:

- on a server to create web applications. [web development (server-side)]
- alongside software to create workflows. [software development]
- connect to database systems. It can also read and modify files.
- to handle big data and perform complex mathematics.
- for rapid prototyping, or for production-ready software development.

Uniqueness

- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Data types in Python relevant to data science

Different Kinds of Python Data Types

- Numeric data types: int, float, complex.
- String data types: str.
- Sequence types: list, tuple, range.
- Binary types: bytes, bytearray, memoryview.
- Mapping data type: dict.
- Boolean type: bool.
- Set data types: set, frozenset.

Data structures are “containers” that organize and group data according to type.

Data structures in Python include list, set, tuples, and dictionary. Each of the data structures is unique in its own way.

The data structures differ based on mutability and order.

Mutability refers to the ability to change an object after its creation. Mutable objects can be modified, added, or deleted after they’ve been created, while immutable objects cannot be modified after their creation.

Order, in this context, relates to whether the position of an element can be used to access the element.

Lists

A list is defined as an ordered collection of items.

The term “ordered collections” means that each item in a list comes with an order that uniquely identifies them. The order of elements is an inherent characteristic that remains constant throughout the life of the list.

Since everything in Python is considered an object, creating a list is essentially creating a Python object of a specific type. When creating a list, all the items in the list should be put in square brackets and separated by commas to let Python know that a list has been created. A sample list can be written as **List_A = [item 1, item 2, item 3....., item n]**

Nested Lists:

A list can be nested, which means that it can contain any type of object. It can include another list or a sublist – which can subsequently contain other sublists itself. There is no limit to the depth with which lists can be nested. An example of a nested list is as follows:

List_A = [item 1, list_B, item 3....., item n]

Lists are mutable

Because they can be altered even after being created. A user can search, add, shift, move, and delete elements from a list at their own will. When replacing elements in a list, the number of elements added does not need to be equal to the number of elements, and Python will adjust itself as needed.

Tuples

A tuple is a built-in data structure in Python that is an ordered collection of objects. Tuples come with limited functionality. Lists are mutable, whereas tuples are immutable. Tuples cannot be modified, added, or deleted once they've been created. Lists are defined by using parentheses to enclose the elements, which are separated by commas.

The use of parentheses in creating tuples is optional, but they are recommended to distinguish between the start and end of the tuple. A sample tuple is written as follows:

```
tuple_A = (item 1, item 2, item 3,..., item n)
```

Empty and One Single Item Tuple

When writing a tuple with only a single element, the coder must use a comma after the item. This is done to enable Python to differentiate between the tuple and the parentheses surrounding the object in the equation. A tuple with a single item can be expressed: **some_tuple = (item 1,)**. If the tuple is empty, the user should include an empty pair of parentheses : **Empty_tuple= ()**

Why Tuples are Preferred over Lists

Tuples are preferred when the user does not want the data to be modified. Sometimes, the user can create an object that is intended to remain intact during its lifetime. Tuples are immutable, so they can be used to prevent accidental addition, modification, or removal of data.

Also, tuples use less memory, and they make program execution faster than using lists. Lists are slower than tuples because every time a new item is added to the list, all the items must be moved one position to the right to make room for the new item. Tuples are identifiable by Python as immutable objects, and they are used in many applications where data integrity is important.

Sets

A set is defined as a unique collection of unique elements that do not follow a specific order.

Sets are used when the existence of an object in a collection of objects is more important than the number of times it appears or the order of the objects. Sets are mutable – they can be modified, added, replaced, or removed.

A sample set:

```
set_a = {"item 1", "item 2", "item 3", ....., "item n"}
```

Application: When checking whether or not some elements are contained in a set or not.

For example, sets are highly optimized for membership tests.

They can be used to check whether a set is a subset of another set and to identify the relationship between two sets.

Python Functions

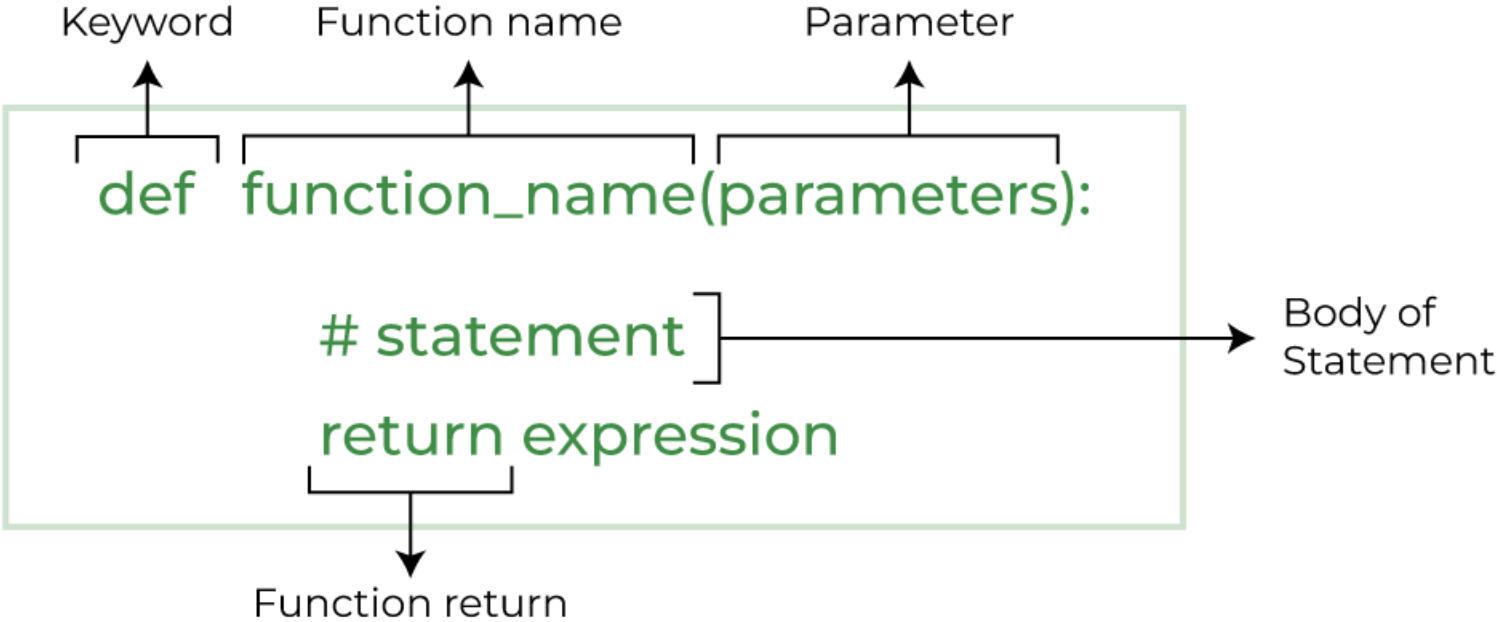
A block of statements that return the specific task.

Put some commonly or repeatedly done tasks together and make a function so that instead of writing the same code again and again for different inputs.

Benefits of Using Functions

- Increase Code Readability
- Increase Code Reusability

Python Function Declaration



Types of Functions in Python

- **Built-in library function:** These are [Standard functions](#) in Python that are available to use.
- **User-defined function:** We can create our own functions based on our requirements.

Def: keyword

A simple Python function

```
def fun():
```

```
    print("Welcome to FAI class")
```

Driver code to call a function

```
fun()
```

Python Function with Parameters

```
def add(num1: int, num2: int) -> int:
```

```
    """Add two numbers"""
```

```
    num3 = num1 + num2
```

```
    return num3
```

```
# Driver code
```

```
num1, num2 = 5, 15
```

```
ans = add(num1, num2)
```

```
print(f"The addition of {num1} and {num2} results {ans}.")
```


Python Function Arguments

A simple Python function to check

whether x is even or odd

```
def evenOdd(x):
```

```
    if (x % 2 == 0):
```

```
        print("even")
```

```
    else:
```

```
        print("odd")
```

Driver code to call the function

```
evenOdd(2)
```

```
evenOdd(3)
```

Types of Python Function Arguments

Python supports various types of arguments that can be passed at the time of the function call.

- **Default argument**
- **Keyword arguments (named arguments)**
- **Positional arguments**
- **Arbitrary arguments** (variable-length arguments `*args` and `**kwargs`)

Default Arguments

A [default argument](#) is a parameter that assumes a default value if a value is not provided in the function call for that argument.

```
# Python program to demonstrate
```

```
# default arguments
```

```
def myFun(x, y=50):
```

```
    print("x: ", x)
```

```
    print("y: ", y)
```

```
# Driver code (We call myFun() with only
```

```
# argument)
```

```
myFun(10)
```

Default Arguments

Output:

x: 10

y: 50

Keyword Arguments :

The idea is to allow the caller to specify the argument name with values so that the caller does not need to remember the order of parameters.

```
# Python program to demonstrate Keyword Arguments
```

```
def student(firstname, lastname):
```

```
    print(firstname, lastname)
```

```
# Keyword arguments
```

```
student(firstname='Geeks', lastname='Practice')
```

```
student(lastname='Practice', firstname='Geeks')
```

Keyword Arguments :

Output:

Geeks Practice

Geeks Practice

Positional Arguments

We used the [Position argument](#) during the function call so that the first argument (or value) is assigned to name and the second argument (or value) is assigned to age. By changing the position, or if you forget the order of the positions, the values can be used in the wrong places, as shown in the Case-2 example below, where 27 is assigned to the name and Suraj is assigned to the age.

```
def nameAge(name, age):  
    print("Hi, I am", name)  
    print("My age is ", age)  
  
# You will get correct output because the argument is given in order  
  
print("Case-1:")  
  
nameAge("Suraj", 27)  
  
# You will get incorrect output because  
# argument is not in order  
  
print("\nCase-2:")  
  
nameAge(27, "Suraj")
```

Positional Arguments

Output:

Case-1:

Hi, I am Suraj

My age is 27

Case-2:

Hi, I am 27

My age is Suraj

Arbitrary Keyword Arguments

In Python Arbitrary Keyword Arguments, [*args, and **kwargs](#) can pass a variable number of arguments to a function using special symbols. There are two special symbols:

- *args in Python (Non-Keyword Arguments)
- **kwargs in Python (Keyword Arguments)

Example 1: Variable length non-keywords argument

Python program to illustrate

*args for variable number of arguments

```
def myFun(*argv):
```

```
    for arg in argv:
```

```
        print(arg)
```

```
myFun('Hello', 'Welcome', 'to', 'GeeksforGeeks')
```

Arbitrary Keyword Arguments

Output:

Hello

Welcome

to

GeeksforGeeks

Variable length keyword arguments

```
def myFun(**kwargs):  
    for key, value in kwargs.items():  
        print("%s == %s" % (key, value))
```

Driver code

```
myFun(first='Geeks', mid='for', last='Geeks')
```

Variable length keyword arguments

Output:

first == Geeks

mid == for

last == Geeks

Nested Function

```
# Python program to
# demonstrate accessing of
# variables of nested functions
def f1():
    s = 'I love cricket'
    def f2():
        print(s)
    f2()
# Driver's code
f1()
```

Output:

I love cricket

Anonymous Functions in Python

Function is without a name

def keyword is used to define the normal functions and the lambda keyword is used to create anonymous functions.

```
def cube(x): return x*x*x
```

```
cube_v2 = lambda x : x*x*x
```

```
print(cube(7))
```

```
print(cube_v2(7))
```

Output:

343

49

Introduction to libraries: NumPy, Pandas, Matplotlib

NumPy

NumPy is a package for scientific computing.

1. Multi dimensional array
2. Methods for processing arrays
3. Element by element operations
4. Mathematical operations like logical, Fourier transform, shape manipulation, linear algebra and random number generation

```
import numpy as np
```


Ndarray - NumPy Array

The ndarray is a multi-dimensional array object consisting of two parts -- the actual data, some metadata which describes the stored data. They are indexed just like sequence are in Python, starting from 0

1. Each element in ndarray is an object of data-type object called dtype
2. An item extracted from ndarray, is represented by a Python object of an array scalar type

Single Dimensional Array ¶

Creating a Numpy Array

```
# Creating a single-dimensional array
```

```
a = np.array([1,2,3]) # Calling the array function
```

```
print(a)
```

Output:

```
[1 2 3]
```

```
# Creating a multi-dimensional array
```

```
# Each set of elements within a square bracket indicates a row
```

```
# Array of two rows and two columns
```

```
b = np.array([[1,2], [3,4]])
```

```
print(b)
```

Output:

```
[[1 2]
```

```
[3 4]]
```

```
# Creating an ndarray by wrapping a list
```

```
list1 = [1,2,3,4,5] # Creating a list
```

```
arr = np.array(list1) # Wrapping the list
```

```
print(arr)
```

Output:

```
[1 2 3 4 5]
```

```
# Creating an array of numbers of a specified range
```

```
arr1 = np.arange(10, 100) # Array of numbers from 10 up to and excluding 100
```

```
print(arr1)
```

Output:

```
[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
```

```
34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57
```

```
58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81
```

```
82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99]
```

```
# Creating a 5x5 array of zeroes
```

```
arr2 = np.zeros((5,5))
```

```
print(arr2)
```

Output:

```
[[0. 0. 0. 0. 0.]
```

```
[0. 0. 0. 0. 0.]
```

```
[0. 0. 0. 0. 0.]
```

```
[0. 0. 0. 0. 0.]
```

```
[0. 0. 0. 0. 0.]]
```

```
# Creating a linearly spaced vector, with spacing
```

```
vector = np.linspace(0, 20, 5) # Start, stop, step
```

```
print(vector)
```

Output:

```
[ 0.  5. 10. 15. 20.]
```

```
# Creating Arrays from Existing Data
```

```
x = [1,2,3]
```

```
# Used for converting Python sequences into ndarrays
```

```
c = np.asarray(x) #np.asarray(a, dtype = None, order = None)
```

```
print(c)
```

Output:

```
[1 2 3]
```

```
# Converting a linear array of 8 elements into a 2x2x2 3D array
```

```
arr3 = np.zeros(8) # Flat array of eight zeroes
```

```
arr3d = arr3.reshape((2,2,2)) # Restructured array
```

```
print(arr3d)
```

Output:

```
[[[0. 0.]
```

```
 [0. 0.]]
```

```
[[0. 0.]
```

```
 [0. 0.]]]
```

Pandas

Pandas is an open-source Python library providing efficient, easy-to-use data structure and data analysis tools. The name Pandas is derived from "Panel Data" - an Econometrics from Multidimensional Data. Pandas is well suited for many different kinds of data:

1. Tabular data with heterogeneously-type columns.
2. Ordered and unordered time series data.
3. Arbitrary matrix data with row and column labels.
4. Any other form observational/statistical data sets. The data actually need not be labeled at all to be placed into a pandas data structure.

Pandas provides three data structure - all of which are build on top of the NumPy array - all the data structures are value-mutable

1. Series (1D) - labeled, homogenous array of immutable size
2. DataFrames (2D) - labeled, heterogeneously typed, size-mutable tabular data structures
3. Panels (3D) - Labeled, size-mutable array

Applications using Python libraries



NumPy



SciPy



TensorFlow



Artificial intelligence



Business applications



Pandas



Scikit-learn



Web development



Desktop GUI



Data science



Matplotlib



Game development



Keras



PyTorch



Software development

Applications using Python libraries



Theano



CAD applications



Requests



Scientific and numeric



Reddit



Seaborn



Web scraping



Pybrain



Audio and visual applica

Hands-on practice with NumPy and Pandas.